

The Corporate Heist — Solution Documentation

Team name: KC STUDIO

Members: JAY NEGI, TARUN RUWALI, FAIZAN ALI ANSARI, ANKUSH SINGH RAWAT, KARAN NAYAL

1. Summary

We predict a “core competence” score by training a LightGBM regression model on `train.csv`, using only the signals the insider says the *new* panel still cares about (technical assessment, aptitude, rating, KPIs, awards, training, experience, notice period, expected/current-CTC ratio), and deliberately excluding features tied to the *old* panel’s biases (college prestige, city, company brand, referral channel, degree “properness”). On top of the model’s score we add three rule-based, insider-driven adjustments specific to this cycle — a bonus for sustained public code contributors, a small “old-boys’-network” bonus for a short list of leadership colleges — and apply hard exclusion rules for title-inflated, fabricated, duplicate, and >2-month-notice profiles, before ranking the remaining Vault candidates and taking the top 500.

2. Data cleaning and parsing

Every numeric-looking column was recruiter-typed free text with mixed units and formats, so we wrote a dedicated parser per column (`clean_utils.py`), each returning a single numeric scale:

Column	Messy examples found	Parsed to
<code>total_experience</code>	“8+ yrs”, “>20”, “19 months”, “17.6 years”	years, $e = \text{first_number}(x) \times \begin{cases} /12 & \text{if "month" in } x \\ 1 & \text{otherwise} \end{cases}$
<code>aptitude_score</code>	“5.7/10”, “86%”, “7.3”	0–10 scale, $a = n$ if plain “/10”, $a = n/10$ if %
<code>last_rating</code>	“5/5”, “Outstanding”, “3”, “New joiner - not rated”	1–5 scale via text map (Outstanding→5, Unsatisfactory→1) or first numeral
<code>notice_period</code>	“Immediate”, “2 months”, “Serving notice - 45 days”	days, $d = 0$ if “immediate”, $\times 30$ if “month”, else $d = n$
<code>current_ctc / expected_ctc</code>	“58.3 LPA”, “9.85 Cr”, “₹41,20,000”, “1692000”	lakhs, $c = n \times 100$ if “Cr”, $c = n$ if “LPA”/“lakh”, $c = n/100,000$ if raw rupees ($n \leq 10,000$)
<code>technical_assessment</code>	“49/100”, “0.43”, “85”	0–100 scale, $t = n \times 100$ if $n \leq 1$, else $t = n$
<code>awards</code>	“-”, “0”, “Yes - Spot Award”, “1 award”	count, 0 if “-”/“0”/none, else ≥ 1
<code>kpi_met, overtime_history</code>	“Y”/“yes”/“1”/“TRUE” vs “N”/“no”/“0”/“FALSE”	boolean $\in \{0, 1\}$
<code>public_code_contributions</code>	“~5”, “12 merged PRs”, “not tracked”	numeric count, NaN if “not tracked”

Institute names had heavy spelling/abbreviation drift — e.g. “Indian Institute of Technology Delhi”, “I.I.T. Delhi”, “IIT-D”, “IITB” all referring to two of the IITs — and were canonicalised with regex normalisa-

tion (uppercase, strip punctuation, expand known abbreviations) before any institute-based logic was applied.
We additionally derive:

$$\begin{aligned} \text{years_since_grad} &= 2026 - \text{graduation_year} \\ \text{age_at_grad} &= \text{age} - \text{years_since_grad}, \quad \text{exp_gap} = e - \text{years_since_grad} \\ \text{ctc_ratio} &= \min\left(\frac{\text{expected_ctc (lakhs)}}{\text{current_ctc (lakhs)}}, 10\right) \end{aligned}$$

3. What drives a great hire — your findings

On the Archive, a LightGBM regressor’s split-count feature importances ranked `technical_assessment`, `exp_years`, `aptitude_score` and `ctc_ratio` highest, followed by `training_hours`, `last_rating`, `num_employers`, `last_job_change` and `notice_days` — i.e. demonstrated competence and effort, not pedigree, is what the historical outcome data itself rewards once the pedigree columns are removed from the feature set. We further computed role-normalised z-scores, since a “good” technical-assessment score differs by `applied_role`:

$$x_i^{\text{tech}} = \frac{t_i - \mu_{\text{role}(i)}}{\sigma_{\text{role}(i)} + \varepsilon}, \quad x_i^{\text{apt}} = \frac{a_i - \mu_{\text{role}(i)}}{\sigma_{\text{role}(i)} + \varepsilon}$$

where μ_{role} , σ_{role} are the mean/std of that feature within the candidate’s `applied_role` group and $\varepsilon = 10^{-9}$ avoids division by zero. Adding these two features improved dev recall from 0.380 to 0.393 (below).

Validation. We validated the whole pipeline end-to-end on the Ledger (`dev.csv` + `dev_winners.csv`, a past cycle with disclosed outcomes): score all 2,999 Ledger candidates, take the top 150 (matching the 150 disclosed winners), and measure

$$\text{recall@150} = \frac{|\text{top150} \cap \text{winners}|}{|\text{winners}|} = \frac{|\text{top150} \cap \text{winners}|}{150}$$

Our pipeline scores **recall@150** $\approx$ **0.393** (59/150 correctly recovered), against an expected $\approx$ 0.05 (150/2999) from random ranking — roughly an 8 $\times$ lift, confirming the debiased feature set carries real predictive signal even without any of the old panel’s pedigree cues.

4. The current cycle

The Vault differs from the Archive in exactly the ways the insider’s debrief describes. We did not take the debrief on faith alone — for the two checkable claims below we compared the *rate* of the pattern in `train.csv` against `test.csv` to confirm each is a genuine, deliberately-injected cycle-specific signal rather than generic data noise:

- **Public code contributions** exist only in `test.csv` (absent from `train.csv` entirely, so no historical coefficient can be learned for it). Per the debrief, “a handful means nothing” but “dozens or more” fast-tracks a candidate. We define

$$\text{sustained_coder}_i = \mathbb{1}[\text{code_contrib}_i \geq 24], \quad \text{some_coder}_i = \mathbb{1}[0 < \text{code_contrib}_i < 24]$$

and add these as flat score bonuses (Section 6) rather than model features, since train has no ground truth to fit a coefficient against.

- **Title inflation** (“Head of”/“VP”/“Director”/“Chief” in `current_title` with under 6 years’ experience) occurs in **0 of 20,000** Archive rows but **65 of 10,000** Vault rows — a rate jump from 0% to 0.65% that cannot be explained by noise, confirming it is a planted trap exactly as the debrief warns (“go straight to the bin”). Hard-excluded.
- **Old-panel biases** — metro city, big-brand employer, referral channel, “proper” degree, gap-free CVs, general college prestige — are excluded from the model’s feature set entirely, so the learned score reflects competence signals only, not what the *old* committee used to reward.
- **New-college talent:** since institute is excluded from the score (bar the leadership flag below), a candidate from a college with zero presence in `train.csv` is judged purely on assessment/aptitude/rating/KPIs — implementing “no pedigree, no problem.”

- **Leadership colleges:** the one surviving old-panel habit (“a handful of colleges where Nightingale’s own leadership studied still get a leg up”). The file does not name these colleges, so as our one explicit assumption we flag the four most prominent IITs — Delhi, Bombay, Kanpur, Madras — with a small additive bonus of +0.15.
- **Two-month join constraint:** “anyone who can’t join inside two months is dead to them this cycle.” We exclude any candidate whose parsed `notice_days` > 60

5. Profiles you excluded and why

Exclusion rule	Definition	Vault count
Title-inflated	senior-title keyword $\wedge$ <code>exp_years</code> < 6	65
Fabricated (“too good to add up”)	composite red-flag count ≥ 2 (below)	102
Duplicate person	second+ row in a name/grad-year/institute cluster	125 rows
Long notice period	<code>notice_days</code> > 60	208

Fabrication red-flag count. For each candidate i we sum five binary indicators:

$$\begin{aligned} \mathbf{F}_i = & \mathbf{a}[\text{age_at_grad}_i \notin [18, 32]] + \mathbf{a}[\text{exp_gap}_i > 3] + \mathbf{a}[e_i < 0] \\ & + \mathbf{a}[\text{num_employers}_i > \frac{\max(e_i, 1)}{0.8}] + \mathbf{a}[\text{near_perfect}_i] \end{aligned}$$

where `near_perfecti` fires when *every* one of technical assessment (≥ 95), aptitude (≥ 9.5), rating (≥ 5), KPI-met, and awards (≥ 1) are simultaneously maxed — the “looks amazing and doesn’t add up” pattern the debrief warns about. Candidates with $\mathbf{F}_i \geq 2$ are excluded as likely fabricated.

Duplicate detection. We block candidates by the key (`normalised_name, graduation_year, institute_canon`) any block with more than one row is treated as one real person. Within remaining singletons sharing graduation year and institute, we run a fuzzy pass using `rapidfuzz.fuzz.ratio` on normalised names and merge pairs scoring ≥ 90 . From each resulting cluster only the single highest-`final_score` row is kept, since Nightingale “only hires a person once.”

6. Model and selection procedure

Model. LightGBM regressor, 400 trees, 31 leaves, learning rate 0.03, subsample/colsample 0.8, `min_child_samples=30`, seed 42, trained on `train.csv`’s `post_hire_score` against the debiased feature vector

$$E_i = [t_i, a_i, \tau_i, \text{kpi}_i, \text{awards}_i, \text{trainings}_i, \text{train_hrs}_i, \text{ot}_i, \text{enrolled}_i, e_i, \text{num_emp}_i, \text{ljc}_i, \text{notice}_i, \text{ctc_ratio}_i, x_i^{\text{tech}}, x_i^{\text{apt}}]$$

Predicted scores $\hat{y}_i = f(E_i)$ are standardised across the scoring pool:

$$x_i = \frac{\hat{y}_i - \mu_{\hat{y}}}{\sigma_{\hat{y}}}$$

Final ranking score.

$$\text{final_score}_i = x_i + 0.35 \cdot \text{sustained_coder}_i + 0.10 \cdot \text{some_coder}_i + 0.15 \cdot \text{leadership_college}_i$$

(the two bonus weights on code contributions were chosen so a sustained contributor can overtake roughly the 65th–70th percentile of the core-score distribution alone, matching the debrief’s “fast-tracked, even if the rest of the profile is only good rather than great”; the leadership bonus is deliberately small since it is meant to be a tie-breaker, not a dominant factor).

Selection. Hard filters (Section 5) are applied first; the surviving pool is de-duplicated by cluster (keeping the max-`final_score` row per cluster); the remainder is sorted by `final_score` descending; the top 500 are written out with `rank` 1 (highest score, most confident) through 500.

7. What did not work

- A binary “top-5%-in-train” classifier ($\text{label} = \mathbb{1}[\text{post_hire_score} \geq 95\text{th percentile}]$) gave dev recall@150 ≈ 0.373 , statistically indistinguishable from the regressor (0.387 pre-filter), so we kept the simpler, more interpretable regression.
- A 5-seed bagged ensemble of the regressor gave recall@150 ≈ 0.367 — no improvement over a single well-tuned model — so we dropped it to keep the pipeline simple and fast.

8. External resources and AI tools (mandatory)

- **AI tool:** Claude (Anthropic) was used to help write the cleaning/feature-engineering code, design the fabrication/duplicate-detection rules, and draft this documentation. All modelling choices, thresholds and validation numbers were checked by us against the provided files only; no outputs were taken on faith.
- **Libraries:** pandas, numpy, lightgbm, rapidfuzz — all from the permitted list in Appendix A of the problem statement.
- No external datasets, pretrained models beyond what is permitted, or paid data were used at any stage.

9. How to run

```
cd code/  
python main.py
```

Requires `train.csv`, `dev.csv`, `dev_winners.csv`, `test.csv` in the same working directory. Writes `submission.csv` there. All random seeds fixed at 42. Measured runtime ≈ 4 seconds on 2 CPU cores, 4 GB RAM — well under the 5-minute limit.